

ALL IN ONE FILE FOR PROJECTS EXPLANATION

CIS – Secure File Storage using Hybrid Cryptography

Hybrid Cryptography

Hybrid cryptography combines the strengths of **asymmetric** and

symmetric encryption for secure, efficient data protection.

- **Asymmetric encryption (RSA):** Uses a pair of keys — public and private. Public key encrypts data, private key decrypts it. It is computationally expensive but ideal for securely sharing small pieces of data (like keys).
- **Symmetric encryption (AES):** Uses a single shared key for both encryption and decryption. Much faster and better suited for encrypting large files.

Why Hybrid?

- Encrypt large files directly with AES using a fast symmetric key.
- Use RSA to encrypt the AES session key to securely transmit it.
- The receiver uses RSA private key to decrypt the session key and then AES to decrypt the file.

This approach balances **security** and **performance**.

Important Algorithmic Components and Code Logic

Key Pair Generation

- Two RSA key pairs are created for two *security tiers* (public and private).
- Keys are generated and stored in-memory to avoid file I/O in a serverless or ephemeral environment.

AES Encryption Process

1. Generate a random AES session key (16 bytes).
2. Encrypt the file data with AES in EAX mode (provides confidentiality + integrity).
3. Encrypt the AES session key with recipient's RSA public key.
4. Concatenate encrypted session key, AES nonce, tag, and ciphertext in a structured binary format.

AES Decryption Process

1. Parse the encrypted file to extract encrypted session key, nonce, tag, and ciphertext.
2. Use RSA private key to decrypt the AES session key.
3. Use AES session key and nonce to decrypt and verify ciphertext integrity using tag.

Flask Endpoints and UI

- Single-page UI for file upload, selecting encryption tier (public or private), and choosing operation (encrypt or decrypt).
- File data sent to `/api/encrypt-decrypt` endpoint.
- Server reads input, generates keys if needed, processes encryption or decryption, then sends back a downloadable file.
- Client displays results, base64 content for encrypted files, or decrypted plaintext (where possible), and offers download.

Comparative Table of Key Concepts in This Hybrid Encryption System

Concept	Description	Why It's Used / Benefits
RSA (Asymmetric)	Encrypts the AES session key with public key; decrypts with private key	Secure key exchange without sharing secret over network
AES (Symmetric)	Efficiently encrypts/decrypts actual file data using session key	Fast encryption for large data with integrity checking
EAX Mode (AES)	AES encryption mode providing confidentiality + authentication tag	Ensures data is not tampered with (integrity verified)
In-memory Key Storage	Keys are generated and stored in-memory (no disk writes)	Suitable for serverless/cloud environments; better security
Base64 Display (Client Side)	Allows encrypted binary data to be displayed as text in the UI	User-friendly encrypted content preview
Flask + CORS	Web server + cross-origin support for frontend-backend communication	Enables REST API for modern web apps
Structured Binary Format	Encryption output combines RSA and AES parts in order with length prefix	Allows reliable parsing and decryption of encrypted data

AOA – Implement Classical Optimization Methods (GD, SA, GA)

This project demonstrates **different optimization algorithms** Gradient Descent, Simulated Annealing, and Genetic Algorithm and compares how each of them tries to minimize the same function.

Gradient Descent (GD)

Think of this as **rolling a ball down a hill**. The function surface is like a landscape. The ball follows the slope (gradient) to reach the valley (minimum).

- It uses derivative information to know in which direction the slope goes down.
- Works well for smooth and convex problems.
- Can get stuck in local minima for non-convex functions.

Simulated Annealing (SA)

Inspired by **metallurgy (cooling molten metal)**. The algorithm begins with high energy (high randomness) and gradually lowers it.

- It accepts not only better moves (downhill) but occasionally worse moves (uphill).
- This allows it to escape local minima and potentially find a better global solution.
- The probability of accepting worse moves decreases as temperature "cools."

Genetic Algorithm (GA)

Inspired by **biological evolution**. It simulates natural selection:

- A *population* of candidate solutions is maintained.
- The fittest survive and reproduce (selection).
- New solutions are hybrids of parents (crossover).
- Random small changes may occur (mutation).
- Over generations, the population evolves towards better solutions.

Code Explanation (Important Algorithm Parts Only)

Gradient Descent

```
grad = gradient(x)
```

```
x = x - lr * grad
```

- `grad = gradient(x)` calculates the slope of the function at current point `x`.
- `x = x - lr * grad` moves the solution in the opposite direction of the slope (downhill).
- The `lr` (learning rate) controls step size. Too big → overshoot. Too small → very slow.

Alternative: Use adaptive methods like Adam optimizer (commonly used in machine learning), which adjust the step size dynamically.

Simulated Annealing

```
delta = f(new_x) - f(x)
```

```
if delta < 0 or np.exp(-delta/T) > np.random.rand():
```

```
    x = new_x
```

```
T *= cooling
```

- `delta` is the change in fitness between new and old solutions.
- If the new solution is better (`delta < 0`), accept it.
- If worse, still accept with some probability depending on `T` (temperature).
- `T *= cooling` reduces randomness gradually, making the search more stable over time.

Alternative: Other metaheuristics like Tabu Search, Particle Swarm Optimization, or Ant Colony Optimization.

Genetic Algorithm

```
fitness = -f(population)
```

```
parents = population[np.argsort(fitness)][-pop_size//2:]
```

```
child = (p1 + p2) / 2
```

```
if np.random.rand() < mutation_rate:
```

```
    child += np.random.uniform(-1, 1)
```

- `fitness = -f(population)` converts minimization problem into maximization (since `f` is better when lower).
- `parents = ...` selects the best half of population (natural selection).
- `child = (p1 + p2) / 2` makes new candidate by averaging two parents (crossover).

- **mutation** introduces randomness to maintain diversity and avoid premature convergence.

Alternative: Other evolutionary operators:

- Different selection strategies (roulette wheel, rank-based).
- More complex crossover (single-point, two-point, uniform).
- Different mutation strategies.

These three show a spectrum: **deterministic (GD)** → **probabilistic single-solution (SA)** → **population-based probabilistic (GA)**.

Algorithm	Core Idea	Pros	Cons	Best Use Cases
Gradient Descent (GD)	Move step-by-step downhill using slope (derivative)	- Fast for convex problems - Simple and efficient - Widely used in ML training	- Needs function derivative - Can get stuck in local minima - Sensitive to learning rate choice	Training neural networks, regression, convex optimization problems
Simulated Annealing (SA)	Inspired by cooling metals: sometimes accepts worse move to escape local trap	- Can escape local minima - Works on non-convex landscapes - Simple to implement	- Slower convergence - Performance depends on cooling schedule - Randomness can cause unstable results	Scheduling, Travelling Salesman Problem (TSP), resource allocation
Genetic Algorithm (GA)	Inspired by evolution: population of solutions evolve via selection, crossover, mutation	- Explores global search space - Handles discrete/non-differentiable functions - Avoids local minima better	- Computationally expensive - Requires many evaluations - May converge slowly	NP-hard problems (TSP, knapsack) Feature selection in ML Engineering design optimization

AOA – Metaheuristic for Traveling Salesman

Traveling Salesman Problem (TSP)

TSP is a classic optimization problem where the goal is to find the shortest possible route visiting each city once and returning to the starting city. It is an NP-hard combinatorial problem, meaning finding the exact optimal solution becomes very expensive as the number of cities grows.

Simulated Annealing (SA) for TSP

- SA is a probabilistic technique that explores different paths by making small random changes (swapping segments of the route).
- It can accept worse solutions early on to avoid getting stuck in local optima.
- The chance of accepting worse solutions decreases gradually (temperature cooling), making the search more focused over time.

Genetic Algorithm (GA) for TSP

- GA simulates natural selection with a population of candidate routes.
- Candidates are ranked by fitness (shorter paths are fitter).
- The best routes are selected to reproduce by combining parts of two "parent" routes (crossover).
- Random mutations (swaps in the route) ensure diversity.
- Over generations, the population evolves towards shorter routes.

Important Algorithmic Code Insights

Distance Matrix Calculation

```
def distance_matrix(cities):
```

```
    # Calculates Euclidean distance between all pairs once for efficiency.
```

- Precomputes distances to avoid redundant calculations during optimization.

Simulated Annealing Core

```
a, b = random.sample(range(n), 2)
```

```
new_path = current_path[:]
```

```
new_path[a:b] = reversed(new_path[a:b]) # Reverse city segment to create neighbor
```

```
if new_dist < best_dist or random.random() < math.exp((best_dist - new_dist) / temp):
```

```

current_path = new_path # Accept new path if better or
probabilistically if worse
temp *= alpha # Decrease temperature

```

- The reversal of a segment is a common perturbation in TSP to explore new solutions locally.
- The acceptance criteria allow escaping local optima early in the process.
- Cooling controls exploration-exploitation balance.

Alternative perturbations: swapping two cities, shifting city positions.

Genetic Algorithm Core

```

parents = random.sample(population[:5], 2) # Select top 5 candidates,
pick 2 parents
cut = random.randint(1, n - 2)
child = parents[0][:cut] + [c for c in parents[1] if c not in parents[0][:cut]]
# Ordered crossover

```

```

if random.random() < mutation_rate:

```

```

    a, b = random.sample(range(n), 2)

```

```

    child[a], child[b] = child[b], child[a] # Mutation by swapping cities

```

- Parent selection focuses on top performers to preserve good genes.
- Ordered crossover maintains city order and avoids duplicates for valid TSP paths.
- Mutation prevents premature convergence by adding variability.

Alternative crossover: partially mapped crossover, cycle crossover.

Aspect	Simulated Annealing (SA)	Genetic Algorithm (GA)
Core Idea	Single solution, probabilistically accepts worse states	Population of solutions evolving through selection and crossover
Exploration	Local moves by segment reversal	Global search using recombination and mutation
Handling Local Minima	Escapes via probabilistic acceptance early on	Avoids by population diversity and mutation
Parameter Tuning	Temperature, cooling rate, number of iterations	Population size, mutation rate, generations
Complexity	Lower computational cost (single path)	Higher cost (maintain population)
Solution Quality	Often quickly finds good local optima	Can find better global optima given enough generations
Best for	Moderate-sized TSP, when computation is limited	Larger TSP with more compute for better solutions
Implementation	Simple and easy to tweak	More complex due to population and genetic operators

AOA – Optimization to Logistic Regression Model on Dataset

Hyperparameter Optimization in Machine Learning

- Hyperparameters are settings of a learning algorithm (e.g., regularization strength C in logistic regression) that strongly affect performance but aren't learned from data directly.
- Optimization aims to find the best hyperparameter values to maximize model accuracy (classification) or R^2 score (regression).
- Evaluations are done via cross-validation to estimate generalization performance.

Gradient Descent (GD) for Hyperparameters

- Uses an iterative approach to minimize (or maximize) an objective function by moving hyperparameters in the direction of the steepest improvement.
- Estimates gradient numerically since the objective isn't analytically differentiable.
- Efficient for continuous hyperparameters with smooth response surfaces.

Simulated Annealing (SA) for Hyperparameters

- A probabilistic search technique that explores hyperparameter space globally.
- Randomly perturbs hyperparameters to escape local minima.
- Temperature gradually cools down to reduce random jumps and focus on refinement.

Genetic Algorithm (GA) for Hyperparameters

- Maintains a population of hyperparameter candidates.
- Uses tournament selection and mutation to evolve better solutions over generations.
- Useful when the objective landscape is rugged and complex.

Important Algorithmic Code Insights

Data Preparation & Evaluation Function

- Converts categorical targets to numeric via LabelEncoder.
- Features are one-hot encoded.

- Task type (classification or regression) is inferred by unique target values count.
- Cross-validation (stratified for classification) is used for robust evaluation.
- Evaluation returns negative mean score (loss) for minimization compatibility with optimization methods.

Gradient Descent Core

```
for i in range(len(x)):
    xx = x.copy()
    xx[i] += eps
    grad[i] = (obj(xx) - fx) / eps # Numerical gradient by finite difference
x -= lr * grad # Update parameters against gradient
x = np.clip(x, [0.01], [10]) # Keep hyperparameters within valid range
```

- Approximates gradient since analytic form is unavailable.
- Updates hyperparameter in steps controlled by learning rate.
- Clips hyperparameter between 0.01 and 10 to keep models stable.

Simulated Annealing Core

```
xn = x + np.random.normal(0, 0.5, size=len(x)) # Random perturbation
xn = np.clip(xn, [0.01], [10])
if fn < fx or np.random.rand() < np.exp(-(fn - fx) / T): # Accept if better or probabilistically worse
    x, fx = xn, fn
T *= alpha # Cooling schedule to reduce exploration gradually
```

- Perturbs hyperparameter by Gaussian noise.
- Accepts probability to escape local optima.
- Temperature cools down to fine-tune.

Genetic Algorithm Core

```
for _ in range(pop_size):
    i, j = np.random.randint(0, pop_size, 2)
    parents.append(pop[i] if fitness[i] < fitness[j] else pop[j]) # Tournament selection
children = parents + np.random.normal(0, 0.2, size=parents.shape) # Mutation
children = np.clip(children, [0.01], [10])
# Select best individuals to survive
```

```
idx = np.argsort(comb_fit)[:pop_size]
pop, fitness = combined[idx], comb_fit[idx]
```

- Selects parents favoring better fitness members.
- Mutates with small Gaussian noise.
- Keeps population size constant by choosing the best.

Comparative Table: Optimization Algorithms for Hyperparameter Tuning

Aspect	Gradient Descent (GD)	Simulated Annealing (SA)	Genetic Algorithm (GA)
Core Idea	Uses gradient info to iteratively improve parameters	Random perturbation with probabilistic acceptance	Population evolution via selection and mutation
Search Style	Local, directional search	Global, probabilistic jump search	Global, population-based exploration
Convergence Speed	Fast if smooth objective	Moderate, influenced by cooling schedule	Slower, requires multiple generations
Handling Local Minima	Prone to getting trapped	Can escape via probabilistic acceptance	Uses population diversity and mutation to avoid
Parameter Domain	Continuous and differentiable-like	Continuous, no gradient needed	Continuous, no gradient needed
Complexity	Low computational cost	Moderate	Higher due to population maintenance
Best for	Smooth hyperparameter surfaces	Rugged surfaces with local minima	Complex, multimodal surfaces and discrete spaces
Implementation Complexity	Simple to moderate	Moderate	More complex due to population and genetic ops

AMT – Collaborative–Style Recommendation

Recommendation Systems Overview

Recommendation systems help users discover relevant items or products by analyzing data about items, users, and interactions.

Collaborative Filtering

- Collaborative filtering algorithms recommend items by analyzing user behavior and preferences, identifying users with similar tastes or behaviors, and suggesting items liked by those similar users.
- It relies on collective user interaction data (ratings, clicks, purchases) without requiring detailed item attributes.
- Common approach: calculate similarity (e.g., cosine similarity) between user or item vectors.

Content–Based Filtering

- Uses features or attributes of items (such as brand, color, price) to recommend items similar to the user's past preferences.
- Focuses on item information rather than user behavior data.

Cosine Similarity for Recommendations

- Cosine similarity is a metric to measure how similar two vectors are, based on the cosine of the angle between them.
- Scores range from -1 (opposite) to 1 (identical), with higher scores indicating more similarity.
- In recommendation systems, it helps find nearest neighbors (most similar items or users) for making predictions.

Workflow of the Collaborative–Style Recommendation

- Preprocess and encode categorical features (e.g., brand and color).
- Normalize numeric features (e.g., ratings, price) for uniform scale.
- Calculate similarity scores between a target product and all others based on combined feature vectors.
- Select and recommend the top N products with the highest similarity scores, excluding the queried product itself.

```
def recommend_collaborative(product_index, top_n=5):  
    sim_scores = cosine_similarity([features_scaled[product_index]],  
    features_scaled).flatten()
```

```

sim_indices = sim_scores.argsort()[-top_n-1:-1][::-1]
recs = df.iloc[sim_indices][['name', 'brand', 'colour', 'price',
'avg_rating']].reset_index(drop=True)
# Format output nicely
output = ""
for i, row in recs.iterrows():
    output += f"{i+1}. {row['name']}\n"
    output += f"Brand: {row['brand']}\n"
    output += f"Colour: {row['colour']}\n"
    output += f"Price: {row['price']}\n"
    output += f"Avg. Rating: {row['avg_rating']}\n\n"
return output

```

Important Implementation Components and Code Logic

Component	Description
Data Cleaning	Fill missing values for key features ('brand', 'colour') to handle incomplete dataset records.
Label Encoding	Convert categorical text data (brand, colour) into numeric encoded form for ML compatibility.
Feature Selection	Use relevant numerical and encoded features (rating, price, brand_encoded, colour_encoded).
Normalization	Apply MinMaxScaler to scale features between 0 and 1 for balanced similarity calculations.
Similarity Calculation	Use cosine similarity metric on scaled feature vectors to find closest product neighbors.
Recommendation Function	Given a product index, compute similarity scores and return the nearest top N product matches.
Output Formatting	Format the recommendation results with readable details (name, brand, color, price, rating).

AMT – Content-Based Recommendation

Content-Based Recommendation Overview

Content-based recommendation systems suggest items similar to a given item based on the item's own attributes and textual description rather than user interaction data.

Text Feature Extraction with TF-IDF

- TF-IDF (Term Frequency-Inverse Document Frequency) is a vectorization technique to convert textual data into numerical vectors.
- It weights words based on their frequency in a document (term frequency) and inversely on how common they are across all documents (inverse document frequency).
- This highlights important distinctive words for each item while reducing the impact of common words.

Combining Text Features

- Multiple textual attributes (e.g., product description, brand, color) are combined into one string per item to capture more comprehensive context about each product.
- TF-IDF is applied to this combined text to create item feature vectors representing semantic content.

Similarity Search Using Cosine Similarity

- Cosine similarity computes the similarity between TF-IDF vectors based on the cosine of the angle between them.
- High cosine similarity between items indicates similar content and themes, useful for recommending similar products.

Workflow of the Content-Based Recommendation

- Handle missing values in key text features by filling with empty strings or placeholders.
- Concatenate textual features into a single string per item to feed into TF-IDF.
- Fit TF-IDF vectorizer to transform combined text into vectors.
- Calculate similarity scores between the target item's vector and all other items'.

- Recommend top N most similar items based on descending similarity scores, excluding the item itself.

Important Implementation Components and Code Logic

Component	Description
Data Cleaning	Fill missing values in textual columns ('description', 'brand', 'colour').
Text Feature Combination	Concatenate multiple text columns into a single combined feature string per product.
TF-IDF Vectorization	Convert combined text features into numeric TF-IDF vectors capturing term importance.
Similarity Calculation	Use cosine similarity metric between TF-IDF vectors to find content-wise closest products.
Recommendation Function	Given a product index, compute similarity scores and select top similar product indices.
Output Formatting	Format the recommended products with their names, brands, colors, and prices for display.

Comparison between Collab and Content

Aspect	Collaborative Filtering	Content-Based Filtering
Data Source	User-item interaction data (ratings, clicks, purchases)	Item metadata and features (descriptions, attributes)
Recommendation Basis	Items liked by similar users	Items similar to those the user has liked or interacted with
Personalization	Based on patterns of user groups and collective behavior	Based on individual user's past preferences and item attributes
Novelty of Recommendations	Can suggest new and diverse items unknown to the user	Typically suggests items similar to those already known by user
Cold Start Problem	Struggles with new users or items due to lack of interaction data	Better handling of new items, but new users still problematic
Explainability	Often less interpretable since recommendations come from user behavior patterns	More interpretable with clear reasons based on item traits
Scalability	Can be computationally intensive with large datasets	Generally more scalable with feature vectorization
Feature Engineering	Minimal, works mainly on user interaction data	Requires detailed and structured item features
Diversity	Tends to offer diverse recommendations based on community trends	May offer narrower recommendations due to similarity focus
Usage Scenarios	Popular in media platforms with rich user interaction data like Netflix, Spotify	Suitable in domains with rich item attribute data like e-commerce, books

ML – Reinforcement Learning for some Game

This Flask web app allows users to play Tic-Tac-Toe against AI controlled by three different algorithms:

- **Minimax Algorithm:** A classic game-tree based exhaustive search for an optimal move using recursion and evaluation of winning states.
- **Q-Learning:** A model-free reinforcement learning method that learns an action-value (Q) table mapping board states and moves to expected rewards through exploration and exploitation.
- **Policy Gradient:** A deep reinforcement learning method using a neural network to learn a stochastic policy that maps board states to move probabilities, optimized using policy gradient updates.

Important Components and Code Logic

Game Utilities

- **check_winner(board)**
Checks board for a winner by verifying defined winning combinations or draw state, returning the winner's player id and winning positions.
- **Board representation:**
Uses a list of 9 integers representing cells: 1 for player (X), -1 for AI (O), 0 for empty.

Minimax Algorithm

- **minimax(board, player):**
Recursively explores game tree for the current player, evaluates states based on win/loss/draw, and returns the best score and move for optimal play.
- Ideal for small state spaces with guaranteed optimality but computationally expensive.

Q-Learning Method

- Maintains a **Q-table** (Q dictionary) mapping state strings (board states as strings) to Q-values for each action.
- Chooses moves using an epsilon-greedy strategy balancing exploration (**random choice**) and exploitation (**best Q-value**).

- Updates Q-values using standard Q-learning update rule with learning rate **ALPHA** and discount factor **GAMMA**.
- Q-table is saved and loaded from disk to preserve training progress (`qtable.pkl`).
- `qtrain()` runs training episodes to build the Q-table.

Policy Gradient Deep RL

- **PolicyNet (neural network):**
Two-layer fully connected network that outputs a probability distribution over 9 board positions for moves.
- **Training:**
Runs episodes where actions are sampled from the policy, collects log probabilities and rewards.
- Uses discounted rewards and policy gradient to update the model to maximize expected winning probability.
- Model parameters are saved/loaded from disk (`policy.pth`).
- `policy_train()` runs the training procedure.

Flask Web App and UI

- Provides a simple Tic-Tac-Toe board front-end with clickable cells and a dropdown to select AI type.
- Sends the current board state and AI choice to `/move` endpoint.
- Server computes AI move using selected algorithm and sends back updated board and messages.
- Front-end highlights winning cells, shows messages (win/draw), and allows game reset.

Important Code Lines Explained

Checks all winning combinations to detect a winner or draw.

```
def check_winner(board):
    wins = [(0,1,2),(3,4,5),(6,7,8),(0,3,6),(1,4,7),(2,5,8),(0,4,8),(2,4,6)]
    for a,b,c in wins:
        if board[a]==board[b]==board[c]!=0:
            return board[a], (a,b,c)
    if 0 not in board:
        return 0, ()
    return None, ()
```

Explores all possible moves recursively, scoring outcomes to choose the best move.

```
def minimax(board, player):
    winner, _ = check_winner(board)
    if winner is not None:
        return winner * player, None
    best_score = -float('inf')
    best_move = None
    for i in range(9):
        if board[i]==0:
            board[i] = player
            score, _ = minimax(board, -player)
            score = -score
            board[i] = 0
            if score > best_score:
                best_score = score
                best_move = i
    return best_score, best_move
```

Runs a single training episode updating Q-values for state-action pairs.

```
def qtrain_episode():
    ...
    # Chooses action randomly or greedily by Q-values
    # Updates Q-table entries with learning rate and reward signal
```

Defines the deep learning model architecture.

```
class PolicyNet(nn.Module):
    ...
    # Neural Net approximating policy giving action probabilities for board states
```

Implements policy gradient training procedure.

```
def policy_episode():
    ...
    # Runs one episode sampling moves from policy, collects rewards,
    # then computes discounted reward and applies policy gradient
    update
```

Main web API endpoint for playing moves.

```
@app.route("/move", methods=["POST"])
```

```
def move():
```

```
    ...
```

```
    # Receives board and ai type, calls chosen AI to compute AI move,
```

```
    # updates board and returns state and winner messages
```